Software Engineering Lab

Lab 40

Property-Based Testing

Menemukan Edge-Case Tersembunyi via Universal Invariant & Shrinking



Lukman / lukman-ss

Ilusi Keamanan Example-Based Testing

Contoh statis buatan manusia membatasi cakupan uji pada jalur yang diharapkan.

BIAS PENGEMBANG

Unit test konvensional menguji nilai yang dipikirkan developer (happy path). Contoh hand-picked mengabaikan kombinasi liar, angka negatif, dan float precision.
Test suite hijau 100% code coverage tetap menyisakan blindspot fatal di produksi.

BUKTI LAB

Format naif float lolos 5/5 contoh statis ($1.00, $5.25, $99.99, $10.50, $0.01). Saat diuji invariant properti dengan input acak, 1.000/1.000 input gagal roundtrip.

PARADIGMA PBT

Uji kebenaran universal (invariants) terhadap ratusan input acak tanpa asumsi sempit.

Unit test berbasis contoh (example-based testing) bekerja dengan menegaskan bahwa f(x) == y untuk segelintir sampel statis. Masalah mendasarnya adalah bias konfirmasi: engineer cenderung hanya menguji kombinasi nilai yang telah mereka antisipasi dalam implementasi. Pada dataset kecil atau input tipikal, kode tampak sempurna dan seluruh baris assertions berstatus PASS. Namun ketika berhadapan dengan data acak tak terduga di lingkungan produksi, asumsi tersembunyi runtuh seketika dan memicu kegagalan sistemik yang fatal.

Example-based tests check developer assumptions; property-based tests explore domain boundaries.

Tiga Pilar Siklus Property-Based Testing

Generasi domain, evaluasi invariant universal, dan reduksi counterexample.

Tahap	Tanggung Jawab	Output Sistem
1. Generator	Buat ratusan sampel input acak terdistribusi	Slices, numbers, structs
2. Invariant	Evaluasi properti universal f(x)	Validasi true / false
3. Shrinker	Pangkas kegagalan ke bentuk terkecil	Minimal counterexample (e.g. [-1])

Siklus kerja Property-Based Testing membagi tugas antara komputasi mesin dan perancangan manusia secara elegan. Manusia bertanggung jawab mendefinisikan batasan domain serta relasi invariant matematika yang wajib selalu bernilai benar. Mesin PBT kemudian bertindak sebagai musuh (adversary) yang mengeksekusi ratusan kombinasi data acak melalui generator, memverifikasi invariant pada setiap iterasi, dan segera mengisolasi kegagalan ke bentuk data paling sederhana menggunakan algoritma shrinking deterministik.

Generate broadly, assert universally, and shrink failures ruthlessly to their core.

Empat Pola Invariant Kanonikal

Pola relasional universal untuk membuktikan kebenaran sistem tanpa oracle eksternal.

Pola Invariant	Formula Formal	Contoh Kasus Nyata
Roundtrip	Decode(Encode(x)) == x	Serialisasi JSON, mata uang, kompresi
Idempotence	f(f(x)) == f(x)	Normalisasi teks, merge interval, cleanup
Oracle Equiv	f_robust(x) == f_naive(x)	Verifikasi algoritma teroptimasi vs referensi
Easy-to-Verify	HardToSolve(x) -> EasyVerify(res)	Validasi shortest path, partisi graf

Kunci keberhasilan PBT terletak pada pemilihan pola invariant yang tepat tanpa menulis ulang logika implementasi di dalam assertion test. Pola Roundtrip menjamin integritas transformasi bolak-balik tanpa data corruption. Pola Idempotence memastikan stabilitas state di mana pemanggilan berulang tidak mengubah hasil. Pola Test Oracle membandingkan algoritma berperforma tinggi terhadap implementasi referensi sederhana. Pola Easy-to-Verify memeriksa keabsahan properti solusi akhir dengan biaya komputasi murah.

Invariant patterns are executable mathematical specifications for software behavior.

Kegagalan Presisi Float pada Mata Uang

Representasi pecahan biner IEEE 754 merusak presisi desimal secara laten.

```
type NaiveCurrency struct{}

func (n NaiveCurrency) Format(dollars float64) string {
    return fmt.Sprintf("$%.2f", dollars)
}

func (n NaiveCurrency) Parse(s string) (float64, error) {
    s = strings.TrimPrefix(s, "$")
    return strconv.ParseFloat(s, 64)
}
```

Menyimpan nilai moneter dalam tipe floating-point float64 adalah anti-pattern klasik dalam rekayasa perangkat lunak. Pemformatan dengan sintaks pembulatan desimal string memotong digit pecahan sub-cent tanpa kendali. Lebih buruk lagi, konversi string kembali ke float64 tunduk pada aproksimasi biner IEEE 754. Pada pengujian konvensional dengan nilai bulat rapi, bug ini tidak pernah muncul, menciptakan ilusi bahwa modul akuntansi bekerja normal padahal pembulatan sen terus bocor.

Floating point binary approximations inevitably corrupt fractional decimal values.

internal/currency/currency.go:9–21

PBT Mendeteksi Kebocoran Float 100%

Pemeriksaan Roundtrip Parse(Format(x)) == x menggagalkan seluruh sampel acak.

```
func TestPropertyNaiveCurrencyFails(t *testing.T) {
    naive := NaiveCurrency{}
    prop := func(val NaiveFloat) bool {
        dollars := float64(val)
        formatted := naive.Format(dollars)
        parsed, err := naive.Parse(formatted)
        if err != nil { return false }
        return parsed == dollars // Gagal pada sub-cent!
    }
    err := quick.Check(prop, &quick.Config{MaxCount: 100})
    // quick.Check mengembalikan error counterexample
}
```

Ketika properti roundtrip diuji menggunakan paket standar Go testing/quick, kelemahan implementasi float64 langsung terekspos tanpa ampun. Dari 1.000 iterasi angka acak, seluruh 1.000 input mengalami kegagalan roundtrip karena nilai desimal asli terpotong oleh format dua angka di belakang koma. PBT membuktikan secara empiris bahwa fungsi Parse(Format(x)) bukanlah fungsi identitas bagi domain float64, mematahkan asumsi keamanan yang sebelumnya diberikan oleh unit test statis.

A roundtrip property check exposes latent serialization and truncation flaws immediately.

internal/currency/currency_test.go:37-61

Desain Robust: Integer Cents Representation

Menghilangkan ambiguitas desimal dengan menyimpan nilai moneter sebagai int64.

```
type RobustAmount struct {
    Cents int64
}

func (r RobustAmount) Format() string {
    absCents := r.Cents; sign := ""
    if absCents < 0 { sign = "-"; absCents = -absCents }
    dollars := absCents / 100; cents := absCents % 100
    return fmt.Sprintf("%s$%d.%02d", sign, dollars, cents)
}
```

Solusi rekayasa yang terbukti secara industri untuk domain mata uang adalah merepresentasikan nominal terkecil sebagai bilangan bulat utuh (integer cents bertipe int64). Format visual desimal hanyalah proyeksi tampilan pada lapisan presentasi string, sedangkan operasi aritmatika internal dan penyimpanan data tetap berada di ranah bilangan diskrit. Pendekatan ini secara definitif menutup peluang terjadinya distorsi presisi IEEE 754 dan menjamin determinisme mutlak.

Represent fractional quantities in discrete atomic units to eliminate rounding artifacts.

Custom Generator & Boundary Value Bias

Mengarahkan distribusi acak ke titik-titik rawan: batas nol, negatif, dan ekstrem.

```
func (RobustAmount) Generate(r *rand.Rand, size int) reflect.Value {
    switch r.Intn(5) {
    case 0: cents = 0                          // Boundary zero
    case 1: cents = int64(r.Intn(200) - 100)  // Near-zero negatives
    case 2: cents = int64(r.Intn(1000000))    // Normal positive
    case 3: cents = -int64(r.Intn(1000000))   // Normal negative
    default: cents = r.Int63n(1e12) - 5e11    // Multi-billion bounds
    }
    return reflect.ValueOf(RobustAmount{Cents: cents})
}
```

Generator acak yang murni seragam (uniform distribution) sering kali melewatkan titik-titik krusial tempat bersarangnya bug batas logika. Implementasi generator RobustAmount pada Lab 40 sengaja menyuntikkan bias probabilitas ke arah nilai rawan: batas nol, transisi bilangan negatif kecil di sekitar nol (-100 hingga +99 sen), serta nilai nominal raksasa multi-miliar sen. Generator yang cerdas secara agresif membombardir tepi spektrum domain untuk menguji ketahanan boundary handling.

Biased generators force tests toward dangerous boundary conditions instead of safe averages.

internal/currency/currency_test.go:63–81

1.000 Iterasi Lolos Roundtrip Integer Cents

Membuktikan secara deterministik bahwa ParseRobust(Format(x)) == x.

Metrik Uji	Naive Float64	Robust Integer Cents
Total Iterasi Acak	1.000 iterasi	1.000 iterasi
Tingkat Kelulusan	0% (1.000 gagal)	100% (1.000 lolos)
Cakupan Domain	Terbatas hand-picked	Nol, negatif, triliun sen
Invarian Terbukti	Gagal total	Parse(Format(x)) == x VALID

Dengan beralih ke integer cents dan parser yang presisi, pengujian properti TestPropertyRobustAmountRoundtrip berhasil mengeksekusi 1.000 iterasi berturut-turut tanpa satu pun pelanggaran invariant. Terlepas dari apakah input berupa angka negatif dengan tanda minus, nilai nol bersih, atau nominal transaksi skala enterprise, relasi Parse(Format(x)) == x selalu terpenuhi. Ini membuktikan bahwa arsitektur data baru telah memenuhi kontrak integritas serialisasi secara matematis.

A verified roundtrip invariant proves mathematical isomorphism between data and representation.

Kasus Interval Merge & Asumsi Tersembunyi

Asumsi implisit bahwa urutan input selalu terurut memicu kecacatan algoritma.

```
func NaiveMerge(intervals []Interval) []Interval {
    if len(intervals) == 0 { return nil }
    out := []Interval{intervals[0]}
    for _, iv := range intervals[1:] {
        last := &out[len(out)-1]
        if iv.Start <= last.End {
            if iv.End > last.End { last.End = iv.End }
        } else { out = append(out, iv) }
    }
    return out // BUG: Mengasumsikan input terurut!
}
```

Kelemahan algoritma sering kali berakar pada prakondisi implisit yang tidak dinyatakan secara tegas dalam kontrak kode. Fungsi NaiveMerge dirancang untuk menggabungkan interval waktu yang saling tumpang tindih. Namun, loop perulangan langsung membandingkan elemen berjalan terhadap elemen terakhir dari slice output, mengasumsikan input sudah terurut berdasarkan titik awal (Start). Ketika dipanggil dengan contoh terurut, fungsi bekerja mulus; namun begitu urutan berubah, output menjadi rusak.

Unenforced implicit preconditions are bugs waiting for real-world input to trigger them.

Test Oracle: Membongkar 85% Discrepancy

Membandingkan implementasi naif terhadap implementasi robust secara komparatif.

```
func TestPropertyNaiveMergeFails(t *testing.T) {
    prop := func(ivs IntervalSlice) bool {
        naive := NaiveMerge([]Interval(ivs))
        robust := RobustMerge([]Interval(ivs))
        return intervalsEqual(naive, robust)
    }
    err := quick.Check(prop, &quick.Config{MaxCount: 100})
    // 85 dari 100 input acak menghasilkan ketidaksesuaian!
}
```

Teknik Test Oracle memanfaatkan dua implementasi berbeda untuk input yang sama: f_naive(x) versus f_robust(x). RobustMerge secara eksplisit melakukan pengurutan sort.Slice sebelum proses penggabungan dilakukan. Ketika dievaluasi dengan generator irisan acak, pengujian membongkar bahwa 85 dari 100 input menghasilkan inkonsistensi keluaran antara NaiveMerge dan RobustMerge. Generator acak secara instan mengekspos betapa rapuhnya kode naif di hadapan array yang tidak terurut.

A reference implementation acts as an oracle to evaluate alternative algorithms at scale.

Invariant Idempotence pada Penggabungan Data

Menjalankan operasi berulang pada hasil penggabungan tidak boleh mengubah state.

```
Formula Teoretis: f(f(x)) == f(x)

func TestPropertyRobustMergeIdempotence(t *testing.T) {
    prop := func(ivs IntervalSlice) bool {
        once := RobustMerge([]Interval(ivs))
        twice := RobustMerge(once)
        return intervalsEqual(once, twice)
    }
    quick.Check(prop, &quick.Config{MaxCount: 1000})
}
```

Idempotensi adalah sifat matematika fundamental dalam arsitektur komputasi: f(f(x)) == f(x). Dalam konteks pemrosesan data, menjalankan fungsi penggabungan pada kumpulan interval yang sudah digabungkan wajib menghasilkan output yang sama persis tanpa mutasi tambahan. Pengujian TestPropertyRobustMergeIdempotence memvalidasi 1.000 iterasi irisan acak, membuktikan bahwa operasi RobustMerge bersifat stabil dan aman dari efek samping mutasi berulang.

Idempotence guarantees operational convergence: repetition causes zero state drift.

Invariant Struktural: Non-Overlapping Slices

Setiap pasangan interval berurutan dijamin tidak saling bertumpukan.

```
func TestPropertyRobustMergeNonOverlapping(t *testing.T) {
    prop := func(ivs IntervalSlice) bool {
        merged := RobustMerge([]Interval(ivs))
        for i := 0; i < len(merged)-1; i++ {
            if merged[i].End >= merged[i+1].Start {
                return false // Ada tumpang tindih!
            }
        }
        return true
    }
    quick.Check(prop, &quick.Config{MaxCount: 1000})
}
```

Selain relasi f(f(x)), kebenaran algoritma penggabungan interval harus memenuhi invariant struktural domain: untuk setiap indeks i dan i+1 pada irisan keluaran, titik akhir interval pertama harus lebih kecil dari titik awal interval berikutnya (out[i].End < out[i+1].Start). Pengujian 1.000 iterasi acak memastikan bahwa tidak ada irisan ganda atau fragmen bertumpuk yang lolos, menjamin partisi interval bebas anomali.

Structural invariants verify domain-specific physical constraints on output shapes.

Anatomi Masalah: Kompleksitas Counterexample

Kegagalan awal dari generator acak mengandung derau data masif yang mengaburkan akar masalah.

SAMPLE INITIAL FAILING INPUT

Array Awal (10 elemen acak bernilai besar & campur aduk):
[137, 18, 100, -27, 95, 107, 126, 78, -7, 79]

Pertanyaan Developer saat Melihat Log:

- Apakah bug dipicu oleh angka 137? Atau ukuran array bernilai 10?
- Apakah urutan elemen saling mempengaruhi?
- Elemen mana yang menjadi pemicu minimal pelanggaran invariant?

Ketika mesin PBT mendeteksi pelanggaran invariant pada input acak berukuran besar, output mentahnya sering kali berupa struktur data raksasa yang membingungkan engineer. Pada contoh demo, input yang memicu kegagalan predicate adalah array 10 elemen acak yang bercampur antara ratusan angka positif dan dua bilangan negatif (-27 dan -7). Tanpa mekanisme reduksi, developer akan menghabiskan waktu berjam-jam memilah data mana yang sesungguhnya menjadi biang kerok kegagalan sistem.

Raw random failures contain heavy noise; automated shrinking isolates the minimal signal.

Konsep dan Algoritma Shrinking

Reduksi deterministik dari kegagalan kompleks menuju kasus reproduksi terkecil.

Fase Algoritma	Aksi Reduksi	Target Optimasi
1. Binary Sectioning	Potong separuh kiri / kanan array	Pangkas ukuran O(log N)
2. Element Removal	Hapus elemen individual satu per satu	Eliminasi noise non-kritis
3. Value Reduction	Bagi dua nilai elemen menuju 0	Minimalisasi magnitudo nilai

Algoritma counterexample shrinking dirancang untuk mentransformasikan input kegagalan mentah menjadi *minimal reproducible counterexample*. Algoritma bekerja secara deterministik dalam tiga fase bertingkat: pertama, melakukan pemotongan biner (halving) terhadap panjang array; kedua, mencoba menghapus elemen satu per satu; dan ketiga, memperkecil magnitudo nilai individual mendekati nol. Setiap langkah yang tetap mempertahankan status gagal (FAIL) diadopsi sebagai basis input baru.

Shrinking is binary search applied to bug reproducers: drop halves, drop items, drop magnitude.

internal/shrinker/shrinker.go:24–60

Binary Sectioning: Memangkas Panjang Array

Menguji apakah separuh bagian data masih mempertahankan kegagalan invarian.

```
if len(current) > 1 {
    mid := len(current) / 2
    left := current[:mid]
    if !predicate(left) {
        current = append([]int(nil), left...)
        changed = true; continue
    }
    right := current[mid:]
    if !predicate(right) {
        current = append([]int(nil), right...)
        changed = true; continue
    }
}
```

Fase pertama dari shrinking adalah binary chunking. Algoritma membagi slice menjadi dua bagian (kiri dan kanan), lalu mengevaluasi predicate pada masing-masing belahan. Jika belahan kiri masih memicu kegagalan, separuh bagian kanan langsung dibuang secara permanen. Dalam demo, array 10 elemen terpotong menjadi 5 elemen [137, 18, 100, -27, 95] pada langkah 1, lalu terpotong lagi menjadi 3 elemen [100, -27, 95] pada langkah 3, mempercepat pencarian secara eksponensial.

Halving input payloads quickly eliminates large clusters of irrelevant test data.

Reduksi Elemen & Magnitudo Menuju Nol

Menghapus elemen per satuan dan memperkecil nilai numerik hingga batas minimal.

```
// Coba set elemen menjadi 0 atau bagi dua nilainya
candidates := []int{0, current[i] / 2}
for _, cand := range candidates {
    testSlice := append([]int(nil), current...)
    testSlice[i] = cand
    if !predicate(testSlice) {
        current = testSlice
        changed = true; break
    }
}
```

Setelah panjang array dipangkas hingga tersisa 1 elemen tunggal berisi [-27], algoritma melanjutkan ke strategi ketiga: reduksi nilai numerik. Nilai -27 secara bertahap dibagi dua mendekati nol: -27 -> -13 -> -6 -> -3 -> -1. Setiap langkah diverifikasi apakah invarian masih dilanggar. Ketika mencapai -1, langkah berikutnya yaitu 0 menghasilkan PASS (karena 0 bukan negatif). Algoritma berhenti dan menetapkan [-1] sebagai nilai batas minimal mutlak.

Gradual numerical reduction isolates the exact mathematical threshold of failure.

internal/shrinker/shrinker.go:119–151

Jejak 21 Langkah Menuju Minimal

Bukti log eksekusi nyata dari FindAndShrink pada internal/shrinker.

```
Input Awal (10 elemen): [137 18 100 -27 95 107 126 78 -7 79]
Step  1: remove-right-half         [5 elem] [137 18 100 -27 95] -> FAIL
Step  3: remove-left-half          [3 elem] [100 -27 95]        -> FAIL
Step  5: remove-left-half          [2 elem] [-27 95]            -> FAIL
Step  6: remove-right-half         [1 elem] [-27]               -> FAIL
Step  9: reduce-val to -13         [1 elem] [-13]               -> FAIL
Step 12: reduce-val to -6          [1 elem] [-6]                -> FAIL
Step 15: reduce-val to -3          [1 elem] [-3]                -> FAIL
Step 18: reduce-val to -1          [1 elem] [-1]                -> FAIL
Hasil Akhir (Minimal Counterexample): [-1] (Tepat 21 langkah evaluasi!)
```

Log terminal dari eksekusi demo memperlihatkan transparansi penuh dari proses reduksi data. Dari input awal acak sebanyak 10 elemen yang berantakan, mesin hanya membutuhkan 21 evaluasi predikat untuk mengeliminasi 9 elemen yang tidak relevan serta memperkecil nilai -27 menjadi -1. Hasil akhir [-1] memberikan kejelasan diagnostik instan bagi developer: masalah bukan pada panjang array atau interaksi elemen, melainkan keberadaan bilangan negatif sekecil apa pun.

Shrinking eliminates debugging guesswork by delivering the minimal reproducible unit.

cmd/demo/main.go:160-210

Integrasi Go Standard Library testing/quick

Membangun PBT murni tanpa dependensi eksternal pihak ketiga.

```
cfg := &quick.Config{
    MaxCount: 1000, // Jumlah iterasi acak yang dieksekusi
    Rand:     rand.New(rand.NewSource(seed)), // Determinisme
}

err := quick.Check(func(a RobustAmount) bool {
    return ParseRobust(a.Format()) == a
}, cfg)
if err != nil {
    t.Fatalf("property violation: %v", err)
}
```

Salah satu keunggulan ekosistem Go adalah ketersediaan paket bawaan testing/quick di dalam standard library tanpa memerlukan dependensi pihak ketiga. Paket ini menyediakan fungsi quick.Check yang mengeksekusi properti hingga ribuan kali dengan konfigurasi MaxCount dan seed kustom. Generator bawaan secara otomatis merefleksikan tipe data dasar, sementara interface quick.Generator memungkinkan engineer menyuntikkan generator buatan sendiri untuk tipe data kompleks.

Zero-dependency property testing is already built into the Go standard library.

internal/currency/currency_test.go:85–102

Determinisme, Seeding, & CI/CD Pipeline

Mengelola trade-off antara keacakan eksploratif dan reproduktibilitas pengujian.

Lingkungan	Konfigurasi Seed	Iterasi	Tujuan
Local PR Check	Fixed Seed (deterministik)	100 - 500	Cepat & lolos tanpa flakiness
Nightly CI	Dynamic Time Seed	10.000 - 50.000	Eksplorasi bug liar & boundary
Bug Reproduction	Injeksi Failing Seed	1 run minimal	Reproduksi 100% konsisten

Kritik yang kerap diarahkan pada PBT adalah risiko munculnya flaky tests jika seed acak berubah di setiap run CI. Solusi profesional adalah menerapkan dual-strategy: pada local testing dan PR verification, gunakan seed tetap dengan 100–500 iterasi agar eksekusi berlangsung sub-detik dan stabil. Pada nightly build atau background fuzzer, aktifkan seed dinamis berbasis timestamp dengan 50.000 iterasi. Setiap kali kegagalan ditemukan, log seed wajib dicatat agar developer dapat mereproduksinya secara instan.

Exploratory randomness belongs in nightly CI; deterministic seeds belong in pull requests.

Matriks Komparasi Example vs PBT

Perbedaan mendasar dalam cakupan, bias pengujian, dan kemampuan isolasi bug.

Dimensi Evaluasi	Example-Based Testing	Property-Based Testing
Cakupan Input	Titik diskrit statis (hand-picked)	Spektrum kontinu acak terdistribusi
Bias Pengujian	Tinggi (mengikuti intuisi engineer)	Rendah (adversarial fuzzing)
Bug Boundary	Kerap terlewat (sub-cent, negative)	Terdeteksi via biased generator
Isolasi Bug	Manual via trace & debugger	Otomatis via counterexample shrinking
Tingkat Keyakinan	False confidence (jalur aman)	Calibrated confidence (invarian kokoh)

Perbandingan matriks memperlihatkan pergeseran paradigma kualitas pengujian. Unit test konvensional sangat baik untuk memvalidasi kontrak fungsional spesifik dan regresi skenario bisnis tertentu. Namun, untuk sistem yang menangani kalkulasi finansial, protokol serialisasi, atau manipulasi struktur data kompleks, PBT memberikan jaminan kebenaran yang jauh lebih kokoh melalui eksplorasi batas domain secara otomatis dan reduksi kegagalan yang deterministik.

Use example tests for concrete user flows; use property tests for structural correctness.

Praktik Terbaik & Anti-Pattern PBT

Panduan menghindari jebakan umum dalam mendesain invariant dan generator.

YANG HARUS DIHINDARI

1. Menulis ulang implementasi di dalam assertion test (duplikasi logika kode).
2. Generator terlalu sempit sehingga mengabaikan batas domain ekstrem.
3. Menguji tanpa shrinker sehingga menyulitkan proses debugging hasil gagal.

POLA YANG DIREKOMENDASIKAN

1. Fokus pada sifat relasional: roundtrip, idempotence, oracle, dan simetri.
2. Suntikkan boundary bias: selalu sertakan 0, -1, MaxInt, dan slice kosong.
3. Simpan seed yang gagal ke dalam unit test regresi berbasis contoh permanen.

Kesalahan paling sering terjadi saat mengadopsi PBT adalah kecenderungan developer menduplikasi logika fungsi yang diuji ke dalam assertion test. Praktik ini salah karena jika implementasi memiliki bug logika, assertion duplikat akan ikut menyetujuinya. Fokuslah murni pada sifat relasional independen. Selain itu, pastikan generator selalu mencakup nilai-nilai ekstrem (nol, batas maksimum integer, irisan kosong), dan simpan counterexample yang ditemukan sebagai unit test regresi permanen.

Assert properties of the relationship, never re-implement the algorithm in the test.

Panduan Reproduksi Mandiri & Verifikasi

Seluruh kode Go standar siap dijalankan dan diuji langsung pada terminal Anda.

```
# 1. Masuk ke direktori lab
cd labs/40-property-based-testing

# 2. Jalankan seluruh property & unit test
go test -v ./...

# 3. Verifikasi ketiadaan race condition
go test -race ./...

# 4. Eksekusi CLI interactive demo
go run ./cmd/demo
```

Seluruh implementasi pada Lab 40 dibangun di atas Go 1.22 standar tanpa dependensi luar, menjamin reproduksibilitas penuh di lingkungan macOS, Linux, maupun Windows. Perintah go test -v ./... mengeksekusi suite unit test, uji properti testing/quick, serta algoritma shrinking. Perintah go run ./cmd/demo menampilkan perbandingan interaktif antara kegagalan float, perbandingan oracle interval, dan jejak reduksi 21 langkah secara real-time.

Standard library tests provide reproducible, zero-dependency validation for engineering teams.

Eksperimen kecil, invariant universal terbukti.

Stop guessing edge cases with static examples. Let invariants and shrinking find them for you.

Software Engineering Lab 40 – Property-Based Testing

internal/currency • internal/interval • internal/shrinker
cmd/demo/main.go • README.md • go.mod
Zero third-party dependencies (pure Go 1.22 stdlib)

REPOSITORY

`github.com/lukman-ss/software-engineering-lab`

`labs/40-property-based-testing`

Lukman / lukman-ss

Sumber: commit 7d244968
Test passing & Race-Clean: 7/7 unit & property tests PASS
Invarian terbukti: Roundtrip Currency, Idempotence Interval, dan 21-Step Shrinking